6.8610 Quiz 1 Study Note

Spring 2026

Aggregated from Lectures 1–7 in `6.8610/LectureNotes`

Course Throughline

- **Natural Language Processing (NLP)** = representation + modeling for human language.
- Main course arc:

  LMs → log-linear / embeddings → RNNs / attention
  → Transformers → evaluation → pretraining → scaling

- Repeated theme: many NLP tasks can be reframed as language modeling.
- Repeated warning: low loss or high fluency does *not* imply true understanding, reliability, or usefulness.

Lecture 1: Language Models

Language model (LM):

$$p_\theta(x_{1:T}).$$

- $x_{1:T}$: token sequence $(x_1, \ldots, x_T)$
- T: sequence length
- θ: model parameters

Autoregressive factorization:

$$p_\theta(x_{1:T}) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{1:t-1}).$$

Conditional language modeling:

$$p_\theta(y_{1:m} \mid x_{1:n}) = \prod_{t=1}^{m} p_\theta(y_t \mid y_{1:t-1}, x_{1:n}).$$

- $x_{1:n}$: input sequence
- $y_{1:m}$: output sequence
- n: input length
- m: output length

Maximum likelihood:

$$\hat{\theta} = \arg\max_\theta p_\theta(x_{1:T}).$$

Naive table size:

$$\sum_{t=1}^{T} V^t$$

for vocabulary size V; this is exponential in context length.

Perplexity:

$$\mathrm{PPL}(x_{1:T}) = \exp\left(-\frac{1}{T}\log p_\theta(x_{1:T})\right) = p_\theta(x_{1:T})^{-1/T}.$$

- PPL: per-token perplexity
- **Lower** PPL: better fit to the evaluation distribution

- Interpret perplexity as the model's average branching factor.
- For a uniform distribution over V words, PPL $= V$.
- **Perplexity sanity checks:** PPL > 0 always, and a unigram maximum-likelihood model should outperform a uniform model on its own corpus.
- **Shorter strings are not automatically better.** Autoregressive sentence probability also depends on whether the model decides to stop, so an end-of-sequence choice can make a longer complete sentence more probable than a shorter prefix.
- **Conditional perplexity reaches** 1 **only for deterministic mappings.** English → Englio can be deterministic after pronoun collapse, but Englio → English remains ambiguous.
- Language is hard because it is ambiguous, compositional, socially grounded, and constantly changing.
- Representation and modeling are the two recurring pillars of the course.

Lecture 2: Log-Linear Models and Embeddings

Historical translation framing:

$$p(y \mid x) \propto p(x \mid y)\, p(y).$$

- x: source sentence
- y: target sentence
- $p(y)$: target-language model

Log-linear next-token model:

$$p_\theta(y \mid x) = \frac{\exp(\theta_y^\top \phi(x))}{\sum_{y' \in V} \exp(\theta_{y'}^\top \phi(x))}.$$

- x: context
- y: next token
- $\phi(x)$: fixed feature vector for context x
- θ_y: parameter vector for token y
- V: vocabulary

Single-example log-likelihood:

$$\log p_\theta(y \mid x) = \theta_y^\top \phi(x) - \log Z(x), \qquad Z(x) = \sum_{y' \in V} \exp(\theta_{y'}^\top \phi(x)).$$

- $Z(x)$: partition function that normalizes scores into probabilities

Gradient:

$$\nabla_{\theta_{y_0}} \log p_\theta(y \mid x) = (\mathbf{1}[y = y_0] - p_\theta(y_0 \mid x))\, \phi(x).$$

- y_0: candidate token
- $\mathbf{1}[y = y_0]$: indicator that y_0 is the gold token

Term frequency-inverse document frequency (TF-IDF):

$$\mathrm{TF}(w, d) = \text{count of } w \text{ in } d,$$

$$\mathrm{IDF}(w) = \log \frac{N}{\mathrm{df}(w)}, \qquad \mathrm{TFIDF}(w, d) = \mathrm{TF}(w, d)\, \mathrm{IDF}(w).$$

- d: document
- w: word type
- N: number of documents
- $\mathrm{df}(w)$: number of documents containing w

Skip-gram:

$$p(y \mid x) = \frac{\exp(u_y^\top w_x)}{\sum_{y' \in V} \exp(u_{y'}^\top w_x)}.$$

- w_x: input embedding of center word x
- u_y: output embedding of context word y

Predict context from center word.

Continuous Bag-of-Words (CBOW):

$$W(x) = \frac{1}{|x|} \sum_{x_i \in x} w_{x_i}, \qquad p(y \mid x) \propto \exp(u_y^\top W(x)).$$

- x: context-word set or context window
- $|x|$: number of context words
- $W(x)$: averaged context embedding

Predict target word from neighboring words.

Pointwise mutual information (PMI) view:

$$u_y^\top w_x \approx \log \frac{p(x,y)}{p(x)p(y)} - c.$$

- c: approximately constant offset

- Bag-of-words and CBOW are order-insensitive.
- Reversing the corpus should leave standard CBOW nearly unchanged because the local context multisets are preserved.
- Skip-gram and CBOW learn useful **type-level** embeddings, not contextualized token representations.
- Embedding quality can be checked intrinsically (similarity/analogy resources) or extrinsically (downstream-task performance).
- CBOW and Skip-gram are **not** proper left-to-right language models.
- Neural embedding methods are not guaranteed to beat count-based models on perplexity, because some of them are not autoregressive sequence models at all.
- Embedding dimensions are usually not individually interpretable.
- High dot products reflect co-occurrence structure and similar contexts.
- For two candidates a and b under the same softmax context,

$$\frac{p(a \mid x)}{p(b \mid x)} = \exp(\text{score}(a,x) - \text{score}(b,x)).$$

- The embedding layer is mostly a learned lookup table; the nonlinear hidden layers do the contextual feature combination.

Lecture 3: Feed-Forward Models, RNNs, Seq2Seq, Attention

Key contrast: masked vs autoregressive feed-forward models

- **Masked objective:** predict a hidden token from surrounding context.
- **Autoregressive objective:** predict the next token from left context only.
- Masked objectives can learn representations; autoregressive objectives directly define normalized sequence probabilities.

Basic recurrent neural network (RNN) update:

$$h_t = f(Wx_t + Vh_{t-1} + b_h), \qquad \hat{y}_t = \text{softmax}(Uh_t + b_y).$$

- x_t: input at timestep t
- h_t: hidden state at timestep t
- $\hat{y}_t$: predicted output distribution at timestep t
- $\text{softmax}(z)_j$: normalized exponential,

$$\text{softmax}(z)_j = \frac{e^{z_j}}{\sum_k e^{z_k}},$$

  where z is a vector of logits and j indexes one output coordinate
- W, V, U: learned parameter matrices
- b_h, b_y: bias terms

Cross-entropy (CE) sequence loss:

$$\mathcal{L} = \sum_{t=1}^{T} \text{CE}(y_t, \hat{y}_t), \qquad \text{CE}(y_t, \hat{y}_t) = -\sum_{j \in V} y_{t,j} \log \hat{y}_{t,j}.$$

- y_t: one-hot gold label at timestep t
- $\hat{y}_{t,j}$: predicted probability of vocabulary item j at timestep t
- V: vocabulary

Backpropagation Through Time (BPTT):

$$\frac{\partial \mathcal{L}_T}{\partial h_k} = \frac{\partial \mathcal{L}_T}{\partial h_T} \prod_{t=k+1}^{T} \frac{\partial h_t}{\partial h_{t-1}}.$$

- **Interpretation**: repeated Jacobian products across time can make gradients vanish or explode

Exploding-gradient fix: gradient clipping

$$g \leftarrow g \cdot \min\left(1, \frac{\tau}{\|g\|}\right).$$

- g: gradient vector
- $\|g\|$: gradient norm
- τ: clipping threshold

Seq2Seq encoder-decoder:

$$h_0^{dec} = h_N^{enc}, \qquad h_t^{dec} = f(W_y y_{t-1} + V h_{t-1}^{dec} + b).$$

- h_N^{enc}: final encoder state
- h_t^{dec}: decoder state at timestep t
- y_{t-1}: previous target-token embedding or representation

Attention score and weights:

$$e_{t,i} = \text{score}(h_t^{dec}, h_i^{enc}), \qquad a_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^{N} \exp(e_{t,j})}.$$

- t: decoder timestep
- i: source position
- $e_{t,i}$: compatibility score between decoder step t and source position i
- $a_{t,i}$: normalized attention weight

Context vector:

$$c_t = \sum_{i=1}^{N} a_{t,i} h_i^{enc}.$$

- c_t: context vector, i.e. weighted average of encoder states

RNN emulating a 2-token feed-forward LM:

$$W_{hh} = \begin{bmatrix} 0 & 0 \\ I_d & 0 \end{bmatrix}, \qquad W_{xh} = \begin{bmatrix} W \\ 0 \end{bmatrix}, \qquad f(z) = z.$$

This construction makes

$$h_t = [Wx_{t-1}; Wx_{t-2}],$$

so the RNN can reproduce a 2-token feed-forward LM.

- W_{hh}: hidden-to-hidden transition matrix of the RNN
- W_{xh}: input-to-hidden matrix of the RNN
- W: embedding matrix from the original 2-token feed-forward LM
- I_d: $d \times d$ identity matrix
- 0: zero block of the appropriate shape
- d: embedding dimension
- $f(z) = z$: identity activation function
- x_{t-1}, x_{t-2}: one-hot vectors for the previous two input tokens
- h_t: RNN hidden state storing the embeddings of the two most recent tokens
- $[a; b]$: concatenation of vector a above vector b

Popular attention scoring functions:

- Additive / concat:

$$e_{t,i} = v_a^\top \tanh\Big(W_a[h_t^{dec}; h_i^{enc}]\Big).$$

- Bilinear / general:

$$e_{t,i} = (h_t^{dec})^\top W_a h_i^{enc}.$$

- Dot product:

$$e_{t,i} = (h_t^{dec})^\top h_i^{enc}.$$

- RNN hidden states are **contextualized token embeddings**; word2vec embeddings are static type vectors.
- Teacher forcing: during training, feed the gold previous token to the decoder.
- Truncated BPTT truncates the **backward** pass, not the forward recurrent state.
- RNN strengths: shared weights, variable-length processing, stateful memory.
- RNN weaknesses: slow, hard to parallelize, vanishing/exploding gradients, weak practical long-range memory.
- Long short-term memory (LSTM) networks are a gated repair to vanilla RNNs.
- Attention breaks the single-vector encoder bottleneck and is the bridge to Transformers.

Lecture 4: Transformers

Queries / keys / values:

$$q_i = W_Q x_i, \qquad k_i = W_K x_i, \qquad v_i = W_V x_i.$$

- x_i: representation of token i
- q_i: query vector of token i
- k_i: key vector of token i
- v_i: value vector of token i

Scaled dot-product attention:

$$a_{ij} = \mathrm{softmax}_j\left(\frac{q_i^\top k_j}{\sqrt{d_k}}\right), \qquad z_i = \sum_{j=1}^{n} a_{ij} v_j.$$

- a_{ij}: attention weight from token i to token j
- d_k: key dimension
- n: sequence length
- z_i: new representation for token i

Matrix form:

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

- Q: matrix of token queries
- K: matrix of token keys
- V: matrix of token values

Multi-Head Attention (MHA):

$$\mathrm{head}_h = \mathrm{Attention}(QW_Q^{(h)}, KW_K^{(h)}, VW_V^{(h)}),$$

$$\mathrm{MHA}(Q, K, V) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_H) W_O.$$

- H: number of attention heads
- W_O: output projection matrix

Position-wise feed-forward neural network (FFNN):

$$\mathrm{FFNN}(z_i) = W_2\, \sigma(W_1 z_i + b_1) + b_2.$$

- σ: position-wise nonlinearity

Residual connection:

$$f_{\mathrm{res}}(x) = f(x) + x.$$

Layer normalization:

$$\hat{z}_{t,j} = \frac{z_{t,j} - \mu_t}{\sigma_t + \epsilon}, \qquad y_{t,j} = \gamma_j \hat{z}_{t,j} + \beta_j.$$

- μ_t: mean over features of token t
- σ_t: standard deviation over features of token t
- ϵ: small stability constant
- γ, β: learned scale and shift parameters

Positional encodings:

$$x_i' = x_i + p_i.$$

- p_i: position encoding for token i

Sinusoidal:

$$\mathrm{PE}_{(\mathrm{pos},2i)} = \sin\left(\frac{\mathrm{pos}}{10000^{2i/d_{\mathrm{model}}}}\right),$$

$$\mathrm{PE}_{(\mathrm{pos},2i+1)} = \cos\left(\frac{\mathrm{pos}}{10000^{2i/d_{\mathrm{model}}}}\right).$$

Masked self-attention:

$$\mathrm{MaskedAttention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top + M}{\sqrt{d_k}}\right) V,$$

with $M_{ij} = 0$ if $j \le i$, else $-\infty$.

- M: causal mask preventing token i from attending to future positions $j > i$

Cross-attention:

$$\mathrm{CrossAttention} = \mathrm{softmax}\left(\frac{Q^{dec}(K^{enc})^\top}{\sqrt{d_k}}\right) V^{enc}.$$

- Q^{dec}: queries from decoder states
- K^{enc}: keys from encoder states
- V^{enc}: values from encoder states

BERT objectives:

$$\max_\theta \sum_{t \in \mathcal{M}} \log p_\theta(x_t \mid x_{\setminus \mathcal{M}}), \qquad p_\theta(\texttt{isNext} \mid s_1, s_2).$$

- $\mathcal{M}$: set of masked token positions
- s_1, s_2: sentence pair used in next-sentence prediction

- Three ways to attend:
  - self-attention: one sequence attends to itself;
  - masked self-attention: same, but future positions blocked;
  - cross-attention: decoder attends to encoder states.

- Without positional information, a Transformer encoder behaves like a bag-of-words system up to permutation symmetry.
- Transformer encoder = self-attention + residual/norm + FFNN.
- Transformer decoder = masked self-attention; encoder-decoder models also add cross-attention.
- **BERT** = encoder-only, pretrained with masked language modeling and originally next sentence prediction.
- BERT masking rule: choose 15% of tokens; of those, 80% `[MASK]`, 10% random, 10% unchanged.
- **GPT** = decoder-only autoregressive model using masked self-attention.
- W_Q, W_K, and W_V are shared parameters across all tokens and examples.
- Raw attention scores come from queries against keys, not keys against values.
- Cross-attention is specifically the decoder weighting encoder-side representations.
- Removing the causal mask changes the training objective; it is not harmless just because inference is left-to-right.
- Constraining $W_Q = W_K$ may reduce expressivity, but it does not change the $O(n^2)$ attention cost.
- Learned absolute positional embeddings do not naturally extrapolate to longer sequence lengths than those seen in training.
- Types of learning: supervised, unsupervised / self-supervised, semi-supervised, multi-task, transfer learning.
- Standard attention is $O(n^2)$ in sequence length.

Lecture 5: Evaluation

- **Intrinsic evaluation**: fit to the training-style objective, e.g. held-out log-likelihood or perplexity.
- **Extrinsic evaluation**: performance on an actual downstream task.
- Core warning: perplexity is useful but not sufficient.

Held-out perplexity:

$$\mathrm{PPL} = \exp\left\{-\frac{1}{n}\sum_{(x,y)} \log p_\theta(y \mid x)\right\}.$$

- n: number of evaluation examples or tokens, depending on setup

LM3 counterexample:

$$p_3(x) = \frac{1}{2}p_1(x)$$
$$\implies \quad \frac{1}{n}\sum_x \log p_3(x) = \frac{1}{n}\sum_x \log p_1(x) + \log\frac{1}{2}.$$

A constant log-likelihood penalty can hide catastrophic generation failures.

Bilingual Evaluation Understudy (BLEU):

$$\mathrm{Precision}_n = \frac{\#(n\text{-grams in reference and prediction})}{\#(n\text{-grams in prediction})},$$

$$\mathrm{BLEU} = \mathrm{BP}\cdot\exp\left(\frac{1}{N}\sum_{n=1}^{N}\log \mathrm{Precision}_n\right).$$

- BP: brevity penalty
- N: largest n-gram order used in BLEU

Recall-Oriented Understudy for Gisting Evaluation (ROUGE)-2:

$$\mathrm{ROUGE\text{-}2} = \frac{\#(\text{2-grams in reference and prediction})}{\#(\text{2-grams in reference})}.$$

BERTScore recall-style version:

$$\mathrm{BERTScore}_{\mathrm{recall}} = \frac{1}{|r|}\sum_i \max_j \cos(r_i, p_j).$$

- r_i: reference-token embeddings
- p_j: prediction-token embeddings
- $|r|$: number of reference tokens

Forced-choice accuracy:

$$\mathrm{Accuracy} = \frac{\#(\text{correct})}{\#(\text{examples})}.$$

- Demo $\neq$ solved problem.
- Recognition and generation are different skills.
- Human evaluation is expensive, noisy, and hard to reproduce, but still the gold standard in many settings.
- LM-as-a-judge can work because verification is often easier than generation.
- Common failure modes: Goodhart's Law, Clever Hans effects, leakage, spurious shortcuts, metric overfitting.

Lecture 6: Pretraining and Fine-Tuning

Transfer-learning pipeline:

$$\theta_{\mathrm{PT}} = \arg\max_\theta \sum_{(x,y)\in D_{\mathrm{PT}}} \log p_\theta(y \mid x),$$

$$\theta_{\mathrm{FT}} = \arg\max_\theta \sum_{(x,y)\in D_{\mathrm{FT}}} \log p_\theta(y \mid x) \quad \text{initialized at } \theta_{\mathrm{PT}}.$$

- D_{PT}: pretraining corpus
- D_{FT}: downstream fine-tuning dataset

Gradient-view intuition:

$$\theta' \leftarrow \theta + \nabla_\theta \log p(y_{\mathrm{PT}} \mid x_{\mathrm{PT}}),$$
$$\theta'' \leftarrow \theta' + \nabla_{\theta'} \log p(y_{\mathrm{FT}} \mid x_{\mathrm{FT}}).$$

Pretraining objectives:

- autoregressive next-token prediction:

$$p_\theta(x_{1:T}) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{1:t-1});$$

- masked language modeling:

$$\max_\theta \sum_{t\in\mathcal{M}} \log p_\theta(x_t \mid x_{\setminus\mathcal{M}});$$

- infilling:

$$\max_\theta \log p_\theta(x_{\mathrm{missing}} \mid x_{\mathrm{prefix}}, x_{\mathrm{suffix}});$$

- ranking / discrimination:

$$p_\theta(y = 1 \mid s_1, s_2).$$

Importance weighting for data mixing:

$$w(x) = \frac{\hat{p}_{\mathrm{FT}}(x)}{\hat{p}_{\mathrm{PT}}(x)}.$$

- $w(x)$: weight that upweights pretraining examples resembling the downstream domain

Adversarial reweighting template:

$$\min_{\theta} \max_{q \in \Delta^K} \sum_{k=1}^{K} q_k \big(\mathcal{L}_k(\theta) - \mathcal{L}_k(\theta_{\mathrm{ref}})\big).$$

- Δ^K: simplex over K data subpopulations
- q_k: weight on subpopulation k
- θ_{ref}: reference model

- Pretraining learns general language/world knowledge; fine-tuning steers behavior to tasks or domains.
- Instruction tuning makes a base model respond usefully to prompts.
- Continued pretraining on in-domain unlabeled text is often helpful.
- Synthetic question-answer (QA) or instruction data can help when labels are scarce.
- Parameter-Efficient Fine-Tuning (PEFT) updates only a small subset of parameters.
- Fine-tuning is often **shallow**: it surfaces or reweights capabilities already present from pretraining.
- Refusal is not the same as forgetting; unlearning is fragile.
- Overaggressive fact fine-tuning can increase hallucination.

Lecture 7: Scaling Laws and Emergent Behavior

Validation loss and perplexity:

$$\mathrm{PPL} = e^L.$$

Generic scaling law:

$$L(X) \propto X^{-\alpha}, \qquad 0 < \alpha < 1.$$

Chinchilla-style model:

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}.$$

- N: parameter count
- D: training-token count
- E: irreducible loss floor
- A, B, α, β: fitted scaling-law constants

Dense-Transformer compute approximation:

$$C \approx 6ND.$$

- C: rough dense-Transformer training-compute budget

Under fixed compute:

$$D = \frac{C}{6N}$$

and choose N, D to minimize the predicted loss.

Log-log linearization:

$$L(X) = kX^{-\alpha} \quad \Longrightarrow \quad \log L(X) = \log k - \alpha \log X.$$

- Workflow: train many smaller models, measure loss, fit power laws, extrapolate.
- Kaplan view: bigger models looked especially powerful.
- Chinchilla refinement: compute-optimal training uses a better parameter/data balance.
- In-context learning: adapt from instructions/examples in the prompt, without gradient updates.
- Chain-of-thought reasoning: generate intermediate reasoning steps token by token.
- Emergence may be real, but some "jumps" are metric artifacts.
- Loss and perplexity remain useful but are increasingly weak proxies for reasoning, safety, and tool use.
- Progress now depends on more than pretraining scale: post-training, tool use, inference-time reasoning, and better inductive biases matter.

Likely Quiz Habits

- Be able to write the core equations from memory.
- Be ready to explain *why* a model/objective does or does not define a valid language model.
- Expect compare/contrast questions: CBOW vs RNN, RNN vs Transformer, BERT vs GPT, intrinsic vs extrinsic evaluation, Kaplan vs Chinchilla.
- When asked about limitations, mention order sensitivity, context length, optimization, parallelism, metric mismatch, and data/compute tradeoffs.